

Presenter Q&A Guide

FastAPI Banking API workshop: every question the room is likely to ask, with answers you can say out loud

How to use this guide. Every behaviour marked *verified* was reproduced by running this exact project (FastAPI 0.141, Pydantic 2.13, SQLAlchemy 2.0) with the test client before writing this guide. Level tags: **BASIC** for anyone, **INTERMEDIATE** for curious freshers, **ADVANCED** for experienced attendees.

Contents

1. Big picture and architecture	7 questions
2. Python concepts used in the code	6 questions
3. Setup and running	6 questions
4. Configuration (config.py)	5 questions
5. Database layer and SQLAlchemy	9 questions
6. Schemas and validation (Pydantic)	7 questions
7. Services and repositories	7 questions
8. Routers and HTTP	9 questions
9. Dependency Injection	7 questions
10. main.py and application lifecycle	5 questions
11. Errors and status codes (quick reference)	reference table
12. Production, security and next steps	5 questions
13. Testing	3 questions
Appendix A. Lab answer key	
Appendix B. Transfer feature: full solution (tested)	
Appendix C. Test example with dependency override (tested)	
Appendix D. Handling questions you can't answer	

The project at a glance

```
banking_api/
  .env                DATABASE_URL, APP_NAME
  requirements.txt    fastapi, uvicorn, SQLAlchemy, pydantic, pydantic-settings
  app/
    main.py           FastAPI app, startup create_all, include_router
    config.py          Settings (pydantic-settings)
    database.py        engine, SessionLocal, Base, get_db()
    models/account.py  Account ORM model -> table "accounts"
    schemas/account_schema.py AccountCreate, AccountOut
    repositories/account_repository.py get_by_id, get_all, create, update
    services/account_service.py create/get/deposit/withdraw rules
    routers/account_router.py /accounts endpoints + AmountRequest
```

Request flow: Client -> Router -> Service -> Repository -> SQLite (bank.db)

1. Big picture and architecture

Questions that usually come in the first 10 minutes, when learners see five folders for a small app.

- Q1 Why is such a small app split into so many folders? Couldn't it all live in main.py?** BASIC
- It could, and for a 20-line prototype that is fine. The split exists to teach a pattern that scales: **Router** (HTTP), **Service** (business rules), **Repository** (data access). Each layer changes for a different reason. A new rule touches only the service; a move from SQLite to PostgreSQL touches only the DB URL; a new URL touches only the router.
- Good answer for the room: *"We are learning the structure you'll meet in a 50,000-line codebase, on a project small enough to read in one sitting."*
- Q2 What is the difference between a Router, a Service and a Repository in one sentence each?** BASIC
- Router:** turns an HTTP request into a Python call and the result back into an HTTP response (paths, status codes, validation).
- Service:** decides whether an operation is allowed and what it means (positive amounts, no overdraft).
- Repository:** the only code that runs queries and commits against the database.
- Q3 Is this the same as MVC?** INTERMEDIATE
- Related but not identical. MVC splits an app into Model, View and Controller. Here the *View* is the JSON response (shaped by Pydantic schemas), the *Controller* role is the router, and the *Model* is split into two ideas: the SQLAlchemy model (storage shape) and the service (behaviour). The router/service/repository split is often called a **layered** or **n-tier** architecture; the repository pattern comes from Domain-Driven Design.
- Q4 Can the router call the repository directly and skip the service?** INTERMEDIATE
- Technically yes; Python won't stop you. But then business rules end up in the router, and a second entry point (a CLI, a batch job, a message consumer) has to duplicate them. Rule of thumb used in this project: **routers only talk to services, services only talk to repositories**. For trivial reads (like `get_all_accounts`) the service is a thin pass-through; that is accepted to keep the rule consistent.
- Q5 Why are there two kinds of 'models': `models/account.py` and `schemas/account_schema.py`?** INTERMEDIATE
- They describe different things. The **SQLAlchemy model** `Account` describes the database table (columns, keys, indexes). The **Pydantic schemas** describe the JSON contract with the client. They differ on purpose: `AccountCreate` has only `owner_name`, so a client can never send its own `id` or starting `balance`. If you reused one class for both, you would leak internal columns or accept fields you shouldn't.
- Q6 What is an API, and what makes this one a REST API?** BASIC
- An API is a contract that lets one program ask another to do something. This one is HTTP + JSON. It is *REST-style* because it is organised around a resource (`/accounts`, `/accounts/{id}`), uses HTTP methods meaningfully (GET reads, POST creates or performs an action) and returns standard status codes. Deposit and withdraw are modelled as *action sub-resources* (POST `/accounts/1/deposit`), a common pragmatic choice.

Q7 Why does `bank.db` already contain data when I unzip the project?

BASIC

The file was shipped with the zip after someone ran the app. It contains five sample rows (Abinash 5000.0, Rahul, Arun, Dev, HustlerCoder at 0.0). It is safe to delete; on the next start the app runs `create_all` and a fresh, empty `accounts` table is created. In real projects database files are never committed to Git.

2. Python concepts used in the code

Freshers often understand FastAPI but get stuck on the Python underneath. Keep a REPL open for these.

Q8 What does `yield` do in `get_db()`? Why not `return`?

BASIC

`yield` makes `get_db` a **generator**. FastAPI runs it up to `yield db`, hands the session to the endpoint, and after the endpoint has finished, resumes the generator so the `finally: db.close()` line runs. With `return`, the function would end immediately and nobody would close the session. The `finally` guarantees closing even if the endpoint raised an exception.

```
def demo():
    print("open")
    try:
        yield 42
    finally:
        print("close")

g = demo()
next(g)      # prints open, returns 42
next(g, None) # prints close (generator finishes)
```

Q9 What do the dots in `from ..database import Base` mean?

BASIC

They are **relative imports**. One dot = the current package, two dots = the parent package. `app/models/account.py` lives in package `app.models`, so `..database` means `app.database`. Relative imports only work when the file is imported as part of a package, which is why you must start the app with `uvicorn app.main:app` from the `banking_api` folder.

Q10 Why do I get `ImportError: attempted relative import with no known parent package` when I run `python app/main.py`?

BASIC

Running a file directly makes it the top-level script `__main__` with no package, so `from .routers...` has nothing to be relative to. Start it the intended way: `uvicorn app.main:app --reload`, or `python -m uvicorn app.main:app`. We reproduced this exact error while preparing the session.

Q11 What is the role of the empty `__init__.py` files?

BASIC

They mark a folder as a regular Python **package**, which makes `app.models`, `app.services`, etc. importable and allows relative imports. Python 3 also supports *namespace packages* without `__init__.py`, but having the file is explicit and avoids surprises with tools.

Q12 What is `Optional[Account]` and `List[AccountOut]`?

INTERMEDIATE

Type hints from the `typing` module. `Optional[Account]` means *an Account or None*: `get_by_id` returns `None` when no row matches. `List[AccountOut]` means *a list of AccountOut*. In Python 3.10+ you can write `Account | None` and `List[AccountOut]`. For FastAPI the hint on `response_model=List[AccountOut]` is not just documentation: it drives validation and the OpenAPI schema.

Q13 Why are the dependencies written as default argument values, e.g. `repo:`

INTERMEDIATE

`AccountRepository = Depends()`?

That is FastAPI's convention for declaring a dependency. `Depends()` is a marker object; FastAPI inspects the function signature, finds the marker and supplies the real value at request time. Because it is a *default value*, Python itself would just pass the marker if you called the class manually; that's why in tests you pass the real object explicitly, e.g. `AccountService(repo=fake_repo)`. A modern equivalent is `Annotated[AccountRepository, Depends()]`.

3. Setup and running

Most first-day questions are about environment, not code.

Q14 What exactly does `uvicorn app.main:app --reload` mean?

BASIC

`uvicorn` is an **ASGI server**. `app.main` is the module path (`app/main.py`, imported as part of the `app` package). `:app` is the name of the variable inside that module that holds the `FastAPI()` object. `--reload` watches files and restarts on change; use it only in development.

Q15 Is FastAPI a server? Why do we need uvicorn?

BASIC

No. FastAPI is a framework that produces an ASGI application object. Something has to open a socket, parse HTTP and call that object: that is uvicorn (alternatives: Hypercorn, Granian; Gunicorn with uvicorn workers in production).

Q16 What is `/docs` and where does it come from?

BASIC

FastAPI generates an **OpenAPI** description of your API at `/openapi.json` from your paths, parameters, type hints and Pydantic models. `/docs` is Swagger UI rendering that description; `/redoc` is an alternative viewer. Nothing is written by hand. Note that the `service` parameters do not appear in the docs, because dependencies are not client input.

Q17 Why a virtual environment?

BASIC

It isolates this project's packages from the system Python and from other projects, so versions don't conflict. `requirements.txt` has no pinned versions, so two learners installing on different days may get different versions; in real projects pin them (`fastapi==0.x.y`) or use a lock file.

Q18 I started the server from another folder and all my accounts disappeared. Why?

INTERMEDIATE

The URL `sqlite:///./bank.db` is **relative to the current working directory** of the process, not to the location of `config.py`. Launching from elsewhere creates a new empty `bank.db` there. Also `env_file=".env"` is looked up relative to the working directory. Always start from `banking_api/`, or build an absolute path from `Path(__file__)`.

Q19 How do I see what is actually inside the database?

BASIC

`sqlite3 bank.db "SELECT * FROM accounts;"` and `sqlite3 bank.db .schema` (shows the table and two indexes `ix_accounts_id`, `ix_accounts_owner_name`). Without the `sqlite3` CLI, use Python's built-in `sqlite3` module or a GUI such as DB Browser for SQLite.

4. Configuration (config.py)

Q20 Where does the app name come from: the code or .env?

BASIC

From `.env`. The class default is "Personal Banking API", but `.env` sets `APP_NAME="Personal Banking API (Local Server)"`, and the server prints *Starting Personal Banking API (Local Server)*. Priority in pydantic-settings, highest first: **real environment variable** → **.env file** → **class default**. We verified that `APP_NAME=FromEnv uvicorn ...` wins over `.env`.

Q21 What does `env_ignore_empty=True` do?

INTERMEDIATE

It treats an environment variable set to an empty string as if it were not set. So `APP_NAME=""` in the shell does not blank the name; the value falls back to `.env` (then to the default). Verified during preparation.

Q22 Why is `settings = Settings()` at module level? Isn't a global bad?

INTERMEDIATE

It is a deliberate **module-level singleton**: Python caches imported modules, so the settings are read once and every `from .config import settings` gets the same object. For tests or multiple configurations you can wrap creation in a function decorated with `functools.lru_cache` and inject it with `Depends(get_settings)`, which allows overriding in tests.

Q23 Why is there a `print` in `config.py`? It prints even when I just import something.

BASIC

It shows that module-level code runs at **import time**. It is handy for teaching but in real code use the `logging` module. Also note the comment in `.env` says *Copy to .env*; the file is already named `.env`. Typically you commit a `.env.example` and keep `.env` (with secrets) out of Git.

Q24 How would I switch to PostgreSQL?

INTERMEDIATE

Install a driver (e.g. `psycopg`) and set `DATABASE_URL=postgresql+psycopg://user:pass@host:5432/bank` in the environment. `database.py` already only adds `check_same_thread` for SQLite URLs, so no code change is needed for the engine. You would also move from `create_all` to Alembic migrations and from `Float` to `Numeric`.

5. Database layer and SQLAlchemy

Q25 What is an ORM and why use one?

BASIC

An **Object-Relational Mapper** maps classes to tables and objects to rows. You write `account.balance -= amount; db.commit()` instead of hand-written `UPDATE` statements. Benefits: less boilerplate, protection from SQL injection through parameter binding, database portability. Cost: you must understand sessions and when SQL actually runs.

Q26 What is the difference between `engine`, `SessionLocal` and a `session`?

BASIC

engine: one per application; knows the URL and manages a connection pool. **SessionLocal**: a *factory* configured with that engine. **Session** (`SessionLocal()`): a short-lived unit of work that tracks the objects you load or add and turns changes into SQL on `commit()`. In this app: one session per HTTP request, via `get_db`.

Q27 Why `check_same_thread=False` for SQLite?

INTERMEDIATE

Python's `sqlite3` driver by default refuses to use a connection from a thread other than the one that created it. FastAPI runs plain `def` endpoints (and their sync dependencies) in a **thread pool**, and the connection pool may hand a connection to a different thread, so the check must be disabled. Safety comes from giving each request its own session. The flag is only added when the URL starts with `sqlite`.

Q28 What do `autocommit=False` and `autoflush=False` mean?

INTERMEDIATE

`autocommit=False`: changes are not saved until you call `commit()`, so several changes can be committed atomically (important for a future transfer feature). (In SQLAlchemy 2.0 this is the only mode; the argument is kept for compatibility.) `autoflush=False`: SQLAlchemy won't automatically send pending changes to the DB before each query; they are sent at `commit()` or an explicit `flush()`.

Q29 Why does `create()` call `refresh()` after `commit()`?

INTERMEDIATE

After `commit()` the session expires the object's attributes. `refresh()` reloads the row immediately so that `account.id` (generated by SQLite) and `balance` are present when the router serialises it. Without refresh they would be lazily reloaded on first access anyway while the session is open; the explicit call makes the intent clear and avoids surprises once the session is closed.

Q30 Why does `update()` not call `db.add(account)`?

INTERMEDIATE

The account was loaded through the **same session** in `get_by_id`, so it is already *persistent* and tracked. Changing `account.balance` marks it dirty and `commit()` emits the `UPDATE`. This works because in one request the repository created for the service holds the one session provided by `get_db`.

Q31 What does `index=True` do and why on `owner_name`?

INTERMEDIATE

It creates a database index (`ix_accounts_owner_name`) so lookups/sorting by owner name are faster. The primary key is already indexed by SQLite, so `index=True` on `id` is redundant but harmless. Indexes speed reads and slightly slow writes.

Q32 The code uses `Column(...)` and `db.query(...)`. Is that outdated?

ADVANCED

Both still work in SQLAlchemy 2.x, but they are the 1.x style. The 2.0 style is `class Base(DeclarativeBase), id: Mapped[int] = mapped_column(primary_key=True)` and `db.scalars(select(Account).where(Account.id == account_id)).first()` (or simply `db.get(Account, account_id)`). The concepts are identical, so the classic style is fine for learning.

Q33 Does `Base.metadata.create_all` update my table if I add a column?

ADVANCED

No. It only issues `CREATE TABLE` for tables that don't exist. An existing `accounts` table is left unchanged, so a new column would be missing and queries would fail. In development you can delete `bank.db`; in real projects use **Alembic** migrations.

6. Schemas and validation (Pydantic)

Q34 What happens if the client sends extra fields, like a `balance` when creating an account?

BASIC

They are silently **ignored** (Pydantic's default `extra="ignore"`). We verified: `{"owner_name": "B", "balance": 9999}` creates B with balance 0.0. The service also builds the ORM object with `balance=0.0` explicitly. To reject extra fields with 422, set `model_config = ConfigDict(extra="forbid")`.

Q35 What does `from_attributes=True` on `AccountOut` do?

BASIC

It lets Pydantic build an `AccountOut` by reading **attributes** of an arbitrary object (`account.id`, `account.balance`) instead of requiring a dict. It is required whenever you convert explicitly, e.g. `AccountOut.model_validate(account)` (verified: without it Pydantic raises `model_type ValidationError`). Nuance for sharp learners: with the FastAPI version we tested (0.141), the endpoint *still* worked after removing it, because FastAPI's own response validation reads attributes from the returned object. Keep it anyway: it documents intent and keeps explicit conversions working. In Pydantic v1 this setting was called `orm_mode = True`.

Q36 What does `response_model=AccountOut` add if the function already returns an `Account`?

BASIC

Three things: (1) the return value is **validated and converted** into `AccountOut`; (2) it is **filtered**, so only `id`, `owner_name`, `balance` are sent even if the model gains an internal column like `password_hash`; (3) it documents the response in OpenAPI/Swagger.

Q37 Why is "50.5" (a string) accepted as an amount, but "abc" gives 422?

INTERMEDIATE

Pydantic v2 runs in **lax mode** by default and converts compatible values: a numeric string becomes a float. "abc" cannot be converted, so FastAPI returns 422 with `type: float_parsing`, `loc: ["body", "amount"]`. Use `StrictFloat` or `ConfigDict(strict=True)` to refuse strings. Verified both cases.

Q38 An empty owner name is accepted. How do I stop that?

INTERMEDIATE

`owner_name: str` only checks the type. Add constraints: `owner_name: str = Field(min_length=1, max_length=100)`, and for amounts `amount: float = Field(gt=0)`. Then invalid input produces **422 before any service method runs**. We verified `Field(gt=0)` gives 422 with error type `greater_than`. Keep the service checks too: the service may be called from places other than HTTP.

Q39 Why is `AmountRequest` defined in the router file and not in schemas?

INTERMEDIATE

Probably convenience. It is a request schema like any other and belongs in `schemas/account_schema.py` for consistency. Good discussion prompt: *Where would you move it, and what imports change?* (Answer: move the class, import it in the router; nothing else changes.)

Q40 Why does the API use `float` for money?

INTERMEDIATE

For simplicity. It is wrong for real money: binary floating point can't represent 0.1 exactly. We reproduced it: depositing 0.1 then 0.2 gives a balance of **0.30000000000000004**. Use `Decimal` in Python (`condecimal/Decimal` in Pydantic) and `Numeric(12, 2)` in SQLAlchemy, or store integer paise.

7. Services and repositories

Q41 Why does the service raise `ValueError` instead of `HTTPException`?

BASIC

The service must not know it is behind HTTP. A CLI or background job should be able to call `withdraw()` and get a Python error, not an HTTP concept. The router **translates**: `ValueError` → 400, `None` → 404. In larger apps you define domain exceptions (`InsufficientFundsError`, `AccountNotFoundError`) and register exception handlers in `main.py`.

Q42 Why does 'account not found' use `return None` while other errors raise? INTERMEDIATE

It is a design choice in this code, inherited from `get_by_id` returning `None`. It is slightly inconsistent: two different signalling mechanisms for failures. A cleaner design raises a specific `AccountNotFoundError`. Mention this as an improvement, not a bug.

Q43 Deposit of -5 into a non-existent account 999: 400 or 404? INTERMEDIATE

400 *Deposit amount must be positive*. The service validates the amount before looking up the account. Verified. Order of checks is part of the API's behaviour, so changing it changes client-visible responses.

Q44 Is a withdrawal that brings the balance to exactly 0 allowed? BASIC

Yes. The check is `account.balance - amount < 0`, so 0 passes. Verified: withdrawing 150.5 from 150.5 returns 200 with balance 0.0.

Q45 The router wraps the call in `try/except ValueError` and raises `HTTPException(404)` inside the `try`. Won't the `except` catch it? INTERMEDIATE

No. `HTTPException` is not a subclass of `ValueError`, so it passes through the `except ValueError` clause untouched and FastAPI returns 404. If the handler were `except Exception`, it would be caught and wrongly turned into a 400.

Q46 Is `withdraw` safe if two requests hit the same account at the same moment? ADVANCED

Not fully. It is a **read-modify-write** in Python: both requests may read balance 100, both pass the check for 80, both write 20: a *lost update*, and money is withdrawn twice. Fixes: an atomic SQL update (`UPDATE accounts SET balance = balance - :amt WHERE id = :id AND balance >= :amt`, then check rows affected), row locking with `with_for_update()` on PostgreSQL, or optimistic locking with a version column. SQLite serialises writers, which reduces but does not remove the risk in this pattern.

Q47 How would you add a transfer between two accounts? INTERMEDIATE

Schema: `TransferRequest(to_account_id: int, amount: float)`. Service: validate amount > 0, not the same account, both exist, sufficient funds; subtract and add; **one commit** so both updates are one transaction. Repository: no new method needed. Router: `POST /accounts/{account_id}/transfer` mapping `None` → 404 and `ValueError` → 400. The full working solution (tested) is in Appendix B.

8. Routers and HTTP

Q48 How does FastAPI know that `account_id` comes from the URL and request from the body? BASIC

By the rules: a parameter whose name appears in the path template (`{account_id}`) is a **path** parameter; a parameter typed as a Pydantic `BaseModel` is the **JSON body**; a simple type (`int`, `str`, `float`, `bool`) that is not in the path is a **query** parameter; a parameter with `Depends` is a **dependency**. So `POST /accounts/1/withdraw?amount=5` returns 422 *body field required*: the endpoint expects the amount in the JSON body.

Q49 What does `APIRouter(prefix="/accounts", tags=["Accounts"])` do? BASIC

`prefix` is prepended to every route in the router, so `@router.get("/{account_id}")` becomes `/accounts/{account_id}`. `tags` groups these endpoints under an `Accounts` heading in Swagger. The router is attached to the app with `app.include_router(account_router)` in `main.py`.

Q50 What happens on GET /accounts/abc?

BASIC

FastAPI tries to convert "abc" to `int`, fails, and returns **422** before your function runs: `{"type": "int_parsing", "loc": ["path", "account_id"], ...}`. The `loc` array tells you exactly where the problem is.

Q51 Why does GET /accounts (no trailing slash) return 307?

INTERMEDIATE

The routes are registered as `/accounts/`. Starlette's router has `redirect_slashes=True`, so a request without the slash gets a **307 Temporary Redirect** to `/accounts/`. Browsers and `curl -L` follow it; some HTTP clients don't follow redirects for POST. Solution: be consistent, or register "" instead of "/" as the path.

Q52 Why is creating an account returning 200 instead of 201?

BASIC

The decorator doesn't set `status_code`, and FastAPI's default is 200. REST convention for resource creation is **201 Created**: `@router.post("/", response_model=AccountOut, status_code=201)`. Good small exercise.

Q53 Why are deposit and withdraw POST and not PUT or PATCH?

BASIC

They are **actions** that are not idempotent: sending the same deposit twice deposits twice. PUT should be idempotent (repeatable with the same result) and PATCH is a partial update of fields. POST fits non-idempotent actions. Real payment APIs add an *idempotency key* header to make retries safe.

Q54 What does PUT /accounts/1 return?

INTERMEDIATE

405 Method Not Allowed. The path exists (for GET), but no route handles PUT. Compare with a totally unknown path, which returns 404. Verified.

Q55 Does the order of route declarations matter?

INTERMEDIATE

Yes: FastAPI matches routes in the order they were added. Here `/ {account_id}` and `/` don't clash. It would matter if you added `@router.get("/summary")` **after** `@router.get("/{account_id}")`: `/accounts/summary` would be matched by the first route and fail int conversion with 422. Declare fixed paths before parameterised ones.

Q56 Should these endpoints be async def?

INTERMEDIATE

Not with this code. The repository uses the **synchronous** SQLAlchemy session, which blocks. In a plain `def` endpoint FastAPI runs the function in a thread pool, so blocking is fine. In an `async def` endpoint, blocking calls would freeze the event loop for all requests. Use `async def` only with async libraries end to end (e.g. SQLAlchemy's `AsyncSession` with an async driver).

9. Dependency Injection

The concept most freshers find magical. Slow down here.

Q57 What is dependency injection, in plain words?

BASIC

Instead of a function creating the objects it needs, it **declares** them, and something else (FastAPI) creates and passes them in. `withdraw_money` says *I need an AccountService*; FastAPI sees the service needs a repository, the repository needs a session, the session comes from `get_db()`. It builds all of that per request and cleans up afterwards.

Q58 What does Depends() with no argument do?

INTERMEDIATE

When the argument is omitted, FastAPI uses the **type annotation** as the dependency. So `service: AccountService = Depends()` is shorthand for `Depends(AccountService)`. FastAPI calls the class like a function, inspecting its `__init__` parameters, which may themselves contain `Depends`.

Q59 How many database sessions are opened for one request? INTERMEDIATE

One. FastAPI caches each dependency's value within a single request (`use_cache=True` by default). We measured it: with a counting override, `get_db` was called exactly once per request. Different requests get different sessions.

Q60 When exactly is the session closed? INTERMEDIATE

After the endpoint has returned (or raised), FastAPI resumes the `get_db` generator and its `finally` block runs `db.close()`. Exception paths are included, which is the point of `try/finally`. The precise moment relative to sending the response has changed across FastAPI versions, so don't rely on the session in background work after the response.

Q61 How does DI help testing? INTERMEDIATE

You can replace any dependency without touching the code: `app.dependency_overrides[get_db] = override_get_db` makes every repository use a test database. You can also unit-test the service by constructing it yourself: `AccountService(repo=FakeRepo())`. See Appendix C for a tested example.

Q62 If the request body is invalid and FastAPI returns 422, does a database session still get opened? ADVANCED

Yes. Verified with spies on `get_db` and `AccountService.__init__`: for `POST /accounts/1/deposit {"amount": "abc"}` the log was `get_db → service_init → close`, then 422. FastAPI resolves dependencies and validates request parameters in the same phase; what is skipped is the **endpoint function body**. Cleanup still runs. Practical lesson: keep dependency constructors cheap and free of side effects (no audit writes, no external calls).

Q63 Why not just create the session inside the repository constructor with `SessionLocal()`? ADVANCED

Then nobody would reliably close it, each repository would get its own session (so service and repository might see different transactions), and tests couldn't swap the database. Letting the framework own the session lifecycle is the core benefit of DI.

10. main.py and application lifecycle

Q64 What does `Base.metadata.create_all(bind=engine)` need to work? BASIC

The model classes must have been **imported** first, because defining `class Account(Base)` is what registers the table on `Base.metadata`. In `main.py` this happens indirectly: importing the router imports the service, which imports the repository and `models.account`. We verified that calling `create_all` before importing the model creates zero tables.

Q65 `@app.on_event("startup")` shows a deprecation warning. What is the modern way?

INTERMEDIATE

A **lifespan** context manager passed to the app. Code before `yield` runs at startup, after `yield` at shutdown. Tested with this project:

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)    # startup
    yield
    # shutdown code (close clients, flush logs) goes here

app = FastAPI(title=settings.APP_NAME, lifespan=lifespan)
```

Q66 Why is `include_router` placed at the bottom of `main.py`? Does position matter?

BASIC

It only needs to run before the server starts serving, which is guaranteed because the whole module executes at import. Placing it last is style. Order among multiple `include_router` calls does matter if paths overlap.

Q67 What is the `GET /` endpoint for?

BASIC

A simple health/welcome check showing the app name from settings. Real services add a dedicated `/health` endpoint that also checks the DB, used by load balancers and Kubernetes probes.

Q68 What happens, step by step, when uvicorn starts?

INTERMEDIATE

1) Python imports `app.main`, which imports config (prints *Starting...*), database (creates engine), router → service → repository → model. 2) `app` object is created and the router included. 3) Uvicorn triggers startup: `create_all` makes the table if missing. 4) Server listens on `127.0.0.1:8000`. With `--reload`, a parent watcher process restarts a worker on file changes.

11. Errors and status codes (quick reference)

Verified against the project with FastAPI 0.141 / Pydantic 2.13 / SQLAlchemy 2.0.

Code	Produced by	Example (verified)	Response body
200	Endpoint returned normally	POST /accounts/1/deposit { "amount": 100 }	{ "id": 1, "owner_name": "A", "balance": 100.0 }
400	Router: <code>ValueError</code> → <code>HTTPException(400)</code>	POST /accounts/1/withdraw { "amount": 1000 }	{ "detail": "Insufficient funds" }
400	Amount check runs before lookup	POST /accounts/999/deposit { "amount": -5 }	{ "detail": "Deposit amount must be positive" }
404	Router: service returned <code>None</code>	GET /accounts/999	{ "detail": "Account not found" }
405	No route for that method	PUT /accounts/1	{ "detail": "Method Not Allowed" }
422	Path validation (FastAPI)	GET /accounts/abc	type int_parsing, loc ["path", "account_id"]
422	Body validation (FastAPI)	POST /accounts/ { "owner_name": 123 }	type string_type, loc ["body", "owner_name"]
422	Body missing	POST /accounts/1/withdraw?amount=5	type missing, loc ["body"]
307	Trailing-slash redirect	GET /accounts	Location: /accounts/

Presenter tip. 400 and 404 come from **your** code (router translating service results). 422 comes from **FastAPI** before your endpoint function runs. 405 and 307 come from the **routing layer**. If a learner can say which of the three produced a code, they understand the architecture.

12. Production, security and next steps

Q69 Is this API secure?

BASIC

No: there is **no authentication or authorisation**. Anyone can list every account and withdraw from any account. Next step: OAuth2 password flow with JWT (FastAPI has built-in helpers in `fastapi.security`), then check that the logged-in user owns the account in the service layer.

Q70 What else would you change before production?

INTERMEDIATE

Money as `Decimal/Numeric`; Alembic migrations instead of `create_all`; auth; input constraints with `Field`; domain exceptions + exception handlers; logging instead of `print`; pinned dependencies; `status_code=201` on create; pagination on GET /accounts/; a transactions/audit table; concurrency-safe balance updates; HTTPS and CORS configuration; tests in CI.

Q71 GET /accounts/ returns every row. What is the problem and the fix?

INTERMEDIATE

With a million accounts, it would load all into memory and send a huge response. Add pagination: `def get_all_accounts(skip: int = 0, limit: int = Query(50, le=100), ...)` and `query.offset(skip).limit(limit)` in the repository. These `skip/limit` become **query** parameters automatically.

Q72 Why keep an audit/transactions table in a banking app?

ADVANCED

Currently only the current balance is stored, so you can't explain *how* it got there, reverse a mistake or reconcile. A `transactions` table (`account_id`, `type`, `amount`, `timestamp`) written in the same DB transaction as the balance change gives history and auditability; many systems derive the balance from the ledger itself.

Q73 How would you scale this?

ADVANCED

Move to PostgreSQL (SQLite allows only one writer at a time), run several uvicorn workers behind a load balancer (the app is stateless except for the DB), use a connection pool sized for the workers, and add caching only for read-heavy, non-critical data. Balances should never be served from a stale cache.

13. Testing

Q74 How do I test an endpoint without starting uvicorn?

BASIC

Use `fastapi.testclient.TestClient(app)`: it calls the app in-process. Combine with `app.dependency_overrides[get_db]` to point at an in-memory SQLite database so tests never touch `bank.db`. Full tested example in Appendix C.

Q75 Why `StaticPool` in the test database setup?

INTERMEDIATE

An in-memory SQLite database (`sqlite://`) exists only per connection. `StaticPool` makes all sessions reuse one connection so the tables created in the fixture are visible to the requests.

Q76 What is worth testing first here?

INTERMEDIATE

The rules and the HTTP mapping: create → 200 with balance 0; deposit/withdraw happy path; withdraw beyond balance → 400 *Insufficient funds*; non-positive amount → 400; missing account → 404; invalid id → 422; extra `balance` on create ignored.

Appendix A. Lab answer key

Lab task	Expected result	Why
Deposit 1000, then withdraw 1500	400 <code>Insufficient funds</code>	Service raises <code>ValueError</code> ; router maps to 400
Withdraw exactly the balance	200, balance 0.0	Rule is <code>balance - amount < 0</code> ; zero is allowed
GET <code>/accounts/abc</code>	422, loc ["path", "account_id"]	Path param must convert to int
Deposit -50 into 999	400, not 404	Amount validated before account lookup
Create with <code>"balance": 5000</code>	200, balance 0.0	Extra field ignored; service sets 0.0
<code>curl -i /accounts</code> (no slash)	307 + Location header	Starlette <code>redirect_slashes</code>

Appendix B. Transfer feature: full solution (tested)

This code was added to a copy of the project and passed the test in Appendix C plus transfer checks (A 100 → transfer 40 → A 60, B 40; same account → 400; unknown target → 404).

```
# schemas/account_schema.py
class TransferRequest(BaseModel):
    to_account_id: int
    amount: float

# services/account_service.py (inside AccountService)
def transfer(self, from_id: int, to_id: int, amount: float):
    if amount <= 0:
        raise ValueError("Transfer amount must be positive")
    if from_id == to_id:
        raise ValueError("Cannot transfer to the same account")
    src = self.repo.get_by_id(from_id)
    dst = self.repo.get_by_id(to_id)
    if not src or not dst:
        return None
    if src.balance - amount < 0:
        raise ValueError("Insufficient funds")
    src.balance -= amount
    dst.balance += amount
    self.repo.update(src)      # ONE commit writes BOTH rows atomically
    return src

# routers/account_router.py
@router.post("/{account_id}/transfer", response_model=AccountOut)
def transfer_money(account_id: int, request: TransferRequest,
                   service: AccountService = Depends()):
    try:
        result = service.transfer(account_id, request.to_account_id,
                                   request.amount)
        if not result:
            raise HTTPException(status_code=404, detail="Account not found")
        return result
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
```

Why one commit? Both `src` and `dst` were loaded through the same session, so a single `commit()` writes both UPDATES in one transaction. Committing after the debit and before the credit would risk money disappearing if the process crashed in between.

Appendix C. Test example with dependency override (tested)

Install `pytest` (and `httpx`, required by `TestClient`), save as `banking_api/test_api.py`, run `pytest -q` from `banking_api/`. It never touches `bank.db`.

```
# test_api.py -> run with: pytest -q
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import StaticPool
from app.main import app
from app.database import Base, get_db

engine = create_engine("sqlite://", connect_args={"check_same_thread": False},
                      poolclass=StaticPool)
TestingSession = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def override_get_db():
    db = TestingSession()
    try:
        yield db
    finally:
        db.close()

@pytest.fixture
def client():
    Base.metadata.drop_all(bind=engine)
    Base.metadata.create_all(bind=engine)
    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()

def test_withdraw_insufficient_funds(client):
    acc = client.post("/accounts/", json={"owner_name": "Asha"}).json()
    r = client.post(f"/accounts/{acc['id']}/withdraw", json={"amount": 10})
    assert r.status_code == 400
    assert r.json() == {"detail": "Insufficient funds"}
```

Appendix D. Handling questions you can't answer

- **Park it visibly.** Write it on a *parking lot* slide or whiteboard and promise an answer in the follow-up message. Then actually send it.
- **Turn it into a live experiment.** Most *what happens if...* questions can be answered in 30 seconds in Swagger or a REPL. Running it together teaches more than a confident guess.
- **Separate FastAPI from Python from SQL.** Many confusing questions become simple once you identify which layer they belong to.
- **Point to the source of truth:** the FastAPI docs (fastapi.tiangolo.com), Pydantic docs, and SQLAlchemy 2.0 docs. Avoid answering version-specific questions from memory.